Part 1

Problem 1:

Log Transformation

$s = c \log (1 + r)$

This transformation will present the image's darker sections as lighter. It expands low-intensity sections and compresses the high-intensity ones.

Inverse Log Transformation

$s = c \log^{-1}(r)$

This transformation will darken the image. This is achieved by distorting the image, expanding high-intensity sections and compressing low-intensity sections. As the name suggests, this is the inverse of the previous transformation.

Power Law Transformation

$s = cr^{\gamma}$

As shown in Figure 1, there are two versions of the power law transformation applied to the original image. The gamma values provided were 0.3 and 3. From Figure 1, we can deduce that if the gamma value is less than 1, it will project a log transformation, and when it is more than 1, it behaves like an inverse log transformation.

```
%Problem 1
img = imread('C:\Users\Ansugan\Downloads\Banana-Single.jpg');
i = rgb2gray (img);
imshow(i);

title('Original Image');

i = double(i);

gamma1 = 0.3;
gamma2 = 3.0;

Image1 = 255 * (i / 255).^gamma1;
Image2 = 255 * (i/ 255).^gamma2;

i= uint8(i);
Image1 = uint8(Image1);
Image2 = uint8(Image2);

figure;
subplot(3, 3, 1);
imshow(i);
title ("Original Image");
subplot(3, 3, 2);
imshow(Image1);
title('Power-law Transformation 0.3)');
subplot(3, 3, 3);
imshow(Image2);
title('Power-law Transformation 3.0)');
```

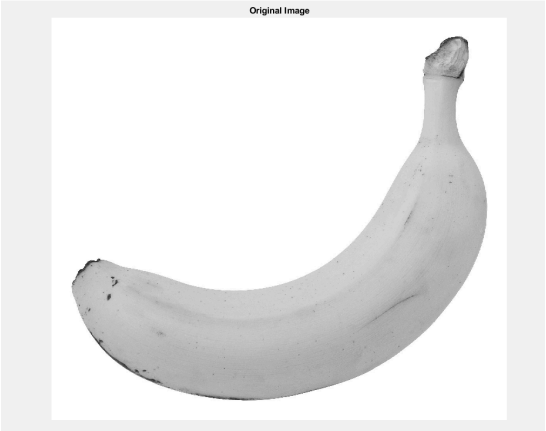




Figure 1: Source Code and Power Log Transformation for Problem 1

Problem 2:

Figure 2 illustrates the 8-bit plane slicing process. We can see that as the bit plane increases, the image decomposes, making it harder to look at each time. From the other figure, we get a closer look at the 2 and 4-bit planes, compared to the original image. It can be observed that the 4-bit image retains more detail than the 2-bit version, appearing sharper and darker.

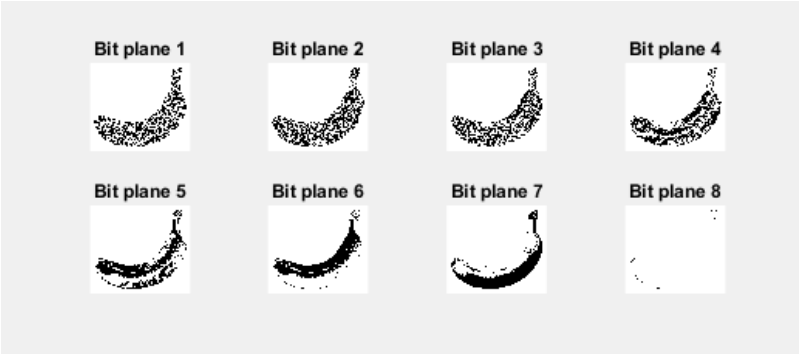
```
%Problem 2

i = rgb2gray(imread('C:\Users\Ansugan\Downloads\Banana-Single.jpg'));

i = double(i);


bitplane1 = bitget(i, 1);
bitplane2 = bitget(i, 2);
bitplane3 = bitget(i, 3);
bitplane4 = bitget(i, 4);
bitplane5 = bitget(i, 5);
bitplane6 = bitget(i, 6);
bitplane7 = bitget(i, 7);
bitplane8 = bitget(i, 8);

figure;
subplot (4, 4, 1);
imshow(bitplane1, []);
title("Bit plane 1");
subplot (4, 4, 2);
imshow(bitplane2, []);
title("Bit plane 2");
subplot (4, 4, 3);
imshow(bitplane3, []);
title("Bit plane 3");
subplot (4, 4, 4);
imshow(bitplane4, []);
title("Bit plane 4");
subplot (4, 4, 5);
imshow(bitplane5, []);
title("Bit plane 5");
subplot (4, 4, 6);
imshow(bitplane6, []);
```



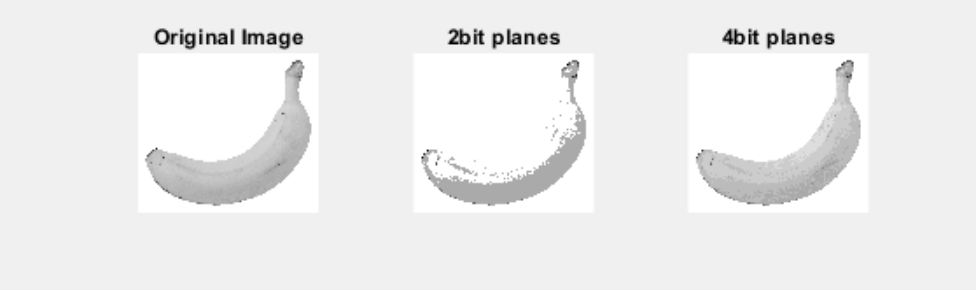


Figure 2: Source Code and 2, 4, and 8-bit planes for Problem 2

Problem 3:

Figure 3 presents multiple images before and after histogram equalization, along with the histograms used in the process. As we can see, the equalized versions on the left appear clearer and more refined when compared to the original. The equalization will redistribute the lower intensity values, enhancing overall contrast. In the image where gamma = 3, the intensity is lower compared to the rest due to its darker appearance; however, after histogram equalization, the histogram effectively spreads out the higher intensity values.

```
%%
%Problem 3

img = imread('C:\Users\Ansugan\Downloads\Banana-Single.jpg');
i = rgb2gray (img);
% imshow(i);
% title('Original Image');

i = double(i);

gamma1 = 0.3;
gamma2 = 3.0;

Image1 = 255 * (i / 255).^gamma1;
Image2 = 255 * (i/ 255).^gamma2;

i= uint8(i);
Image1 = uint8(Image1);
Image2 = uint8(Image2);

orgHist = imhist(i);
hist01 = imhist(Image1);
histo2 = imhist(Image2);

orgHist= uint8(orgHist);
hist01 = uint8(hist01);
histo2 = uint8(histo2);

equalizedImage = histeq(i);
equalizedTransImage1 = histeq(Image1);
equalizedTransImage2 = histeq(Image2);
```

```
figure;
subplot(3, 3, 1);
imshow(i);
title('Grayscale Image');

subplot(3, 3, 2);
bar(orgHist);
title('Histogram (Original)');

subplot(3, 3, 3);
imshow(equalizedImage);
title('Equalized Image (Original)');

subplot(3, 3, 4);
imshow(Image1);
title('Transformed Image (\gamma = 0.3)');

subplot(3, 3, 5);
bar(hist01);
title('Histogram (Transformed, \gamma = 0.3)');

subplot(3, 3, 6);
imshow(equalizedTransImage1);
title('Equalized Image (Transformed, \gamma = 0.3)');

subplot(3, 3, 7);
imshow(Image2);
title('Transformed Image (\gamma = 3.0)');

subplot(3, 3, 8);
bar(histo2);
title('Histogram (Transformed, \gamma = 3.0)');
```

```
subplot(3, 3, 9);
imshow(equalizedTransImage2);
title('Equalized Image (Transformed, \gamma = 3.0)');
```

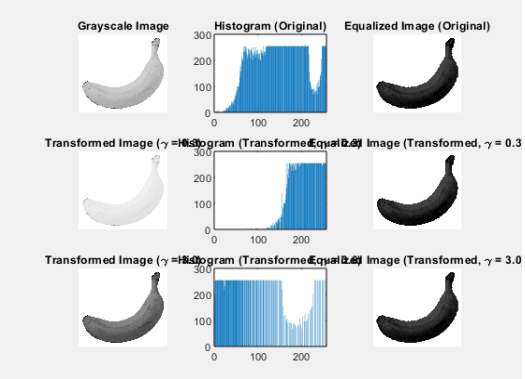


Figure 3: Source Code and Histograms for Problem 3

HW #1

September 21, 2025 9:10 PM

Problem 4:

Problem 4. For the following image demonstrated in class, show the process of histogram equalization, and calculate the probability density function of the equalized image. (4 pts)

r_k	n_k	$p_r(r_k) = n_k/MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02

TABLE 3.1 Intensity distribution and histogram values for a 3-bit, 64×64 digital image.

$$S_k = T(r_k) = (L-1)\sum_{j=0}^{k} P_r(r_j)$$

$S_0 = 7 \cdot P_r(r_0) = 7(0.19) = 1.33$

$S_1 = 7 \cdot [P_r(r_0) + P_r(r_1)] = 7(0.19 + 0.25) = 3.08$

$S_2 = 7 \cdot [P_r(r_0) + P_r(r_1) + P_r(r_2)] = 7(0.19 + 0.25 + 0.21) = 4.55$

$S_3 = 7 \cdot [P_r(r_0) + P_r(r_1) + P_r(r_2) + P_r(r_3)] = 7(0.19 + 0.25 + 0.21 + 0.16) = 5.16$

$S_4 = 7(0.19 + 0.25 + 0.21 + 0.16 + 0.08) = 6.23$

$S_5 = 7(0.19 + 0.25 + 0.21 + 0.16 + 0.08 + 0.06) = 6.65$

$S_6 = 7(0.19 + 0.25 + 0.21 + 0.16 + 0.08 + 0.06 + 0.03) = 6.86$

$S_7 = 7(0.19 + 0.25 + 0.21 + 0.16 + 0.08 + 0.06 + 0.03 + 0.02) = 7$

Histogram Equalization Values:

$S_0 = 1.33 = 1$

$S_1 = 3.08 = 3$

$S_2 = 4.55 = 5$

$S_3 = 5.67 = 6$

$S_4 = 6.23 = 6$

$S_5 = 6.65 = 7$

$S_6 = 6.86 = 7$

$S_7 = 7 = 7$

PDF ($P_s(s_k)$)

r_k	$P_r(r_k)$	S_k	$P_s(s_k)$	
$r_0 = 0$	0.19	0	0	
$r_1 = 1$	0.25	1	0.19	0.19
$r_2 = 2$	0.21	2	0	
$r_3 = 3$	0.16	3	0.25	0.25
$r_4 = 4$	0.08	4	0	
$r_5 = 5$	0.06	5	0.21	0.21
$r_6 = 6$	0.03	6	0.24	$0.16 + 0.08 = 0.24$
$r_7 = 7$	0.02	7	0.11	$0.06 + 0.03 + 0.02 = 0.11$

Problem 5:

Problem 5. For the following 3-bit image, compute the histogram $p_r(r_k)$ of the image and manually perform histogram equalization. Show the computation steps, as well as the histograms before and after transformation. (4 pts)

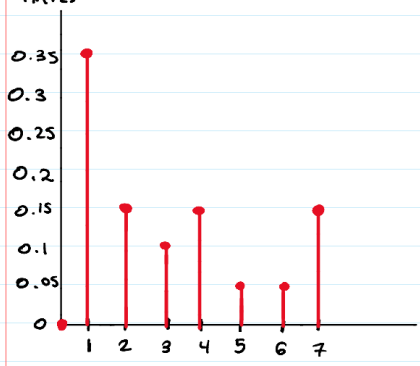
1	2	4	7	3
2	4	7	3	1
5	6	2	1	1
4	7	1	1	1

5 columns, 4 rows
M = 5x4 = 20
8 gray levels

r_k	n_k	$P_r(r_k) = n_k/MN$	$S_k = (L-1)\sum_{i=0}^{k} P_r(r_i)$	round (S_k)	$P_s(s_k)$
0	0	$0/20 = 0$	$7(0/20) = 0$ →	0	0
1	7	$7/20 = 0.35$	$7(7/20) = 2.45$ ↘	1	0
2	3	$3/20 = 0.15$	$7(10/20) = 3.5$ ↘	2	0.35
3	2	$2/20 = 0.10$	$7(12/20) = 4.2$ ↘	3	0
4	3	$3/20 = 0.15$	$7(15/20) = 5.25$ ↘	4	0.25
5	1	$1/20 = 0.05$	$7(16/20) = 5.6$ ↘	5	0.15
6	1	$1/20 = 0.05$	$7(17/20) = 5.95$ →	6	0.10
7	3	$3/20 = 0.15$	$7(20/20) = 7$ →	7	0.15

Histograms:

Before
$P_r(r_k)$



$P_r(r_k)$

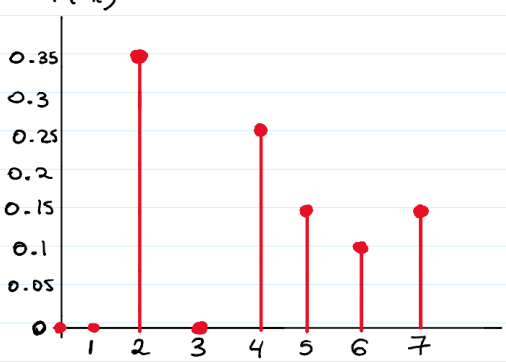


Figure 4: Problems 4 & 5 process and computation

Part 2

Step 1 transforms the image using a simple shear transformation. This transformation shifts a point from the image away from the axis. The image bends the farther it is from the other axis. As can be seen by comparing the original image and the transformed image, the latter is slanted, which is characteristic of the simple shear transformation.

```
%%
%Part 2
%Step 1

a = 0.45;
T = maketform('affine', [1 0 0; a 1 0; 0 0 1] );

A = imread('Soccer_ball_football_general_view.jpg');
h1 = figure; imshow(A); title('Original Image');

% a shade of orange as a filler
orange = [255 127 0]';

R = makeresampler({'cubic','nearest'},'fill');
B = imtransform(A,T,R,'FillValues',orange);
h2 = figure; imshow(B);
title('Sheared Image');
```





Figure 5: Source Code and Sheared Image for Part 2

Step 2 was an instruction on how to plot grid lines onto the sheared image. This observation illustrates how the lines were affected by the basic shear transformation and how they were bent, as shown in the figure below.

```
%Step 2
[U,V] = meshgrid(0:64:320,0:64:256);
[X,Y] = tformfwd(T,U,V);
gray = 0.65 * [1 1 1];

figure(h1);
hold on;
line(U, V, 'Color',gray);
line(U',V','Color',gray);

figure(h2);
hold on;
line(X, Y, 'Color',gray);
line(X',Y','Color',gray);
```

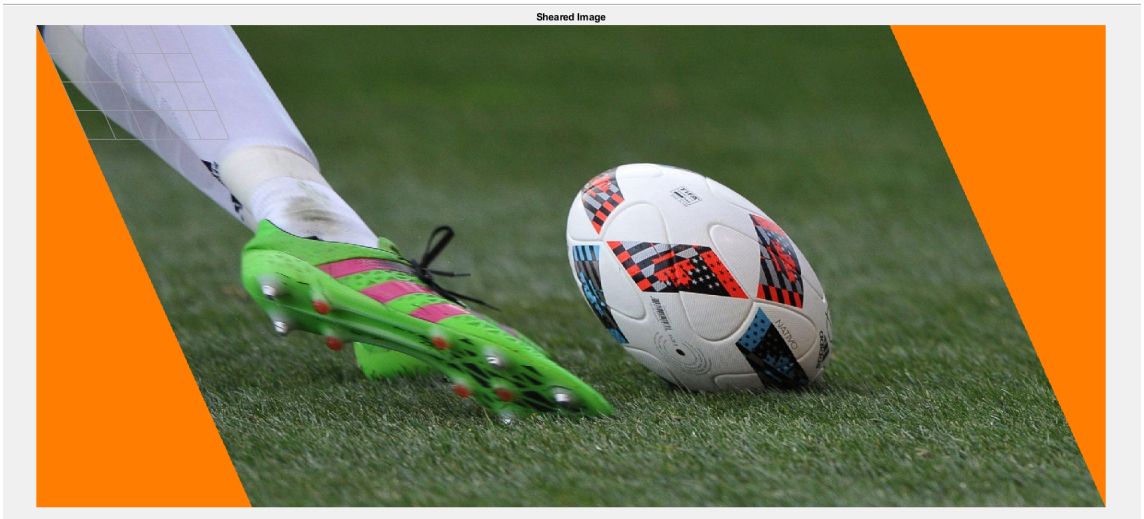


Figure 6: Source Code and Gridded Image for Part 2

For step 3, it was instructed that the image was to go through three padding methods. The first method was the “fill” method, which adds colour to the sheared image’s background. The next pad method used was the “replicate” method. This method duplicates the source image, flips it, and then uses that as the new padding for the image. The final method used was the “bound” method. This method outlines the borders between the padding and the original image.

```
%Step 3
R = makeresampler({'cubic','nearest'},'fill');

Bf = imtransform(A,T,R,'XData',[-49 500],'YData',[-49 400],...
                 'FillValues',orange);

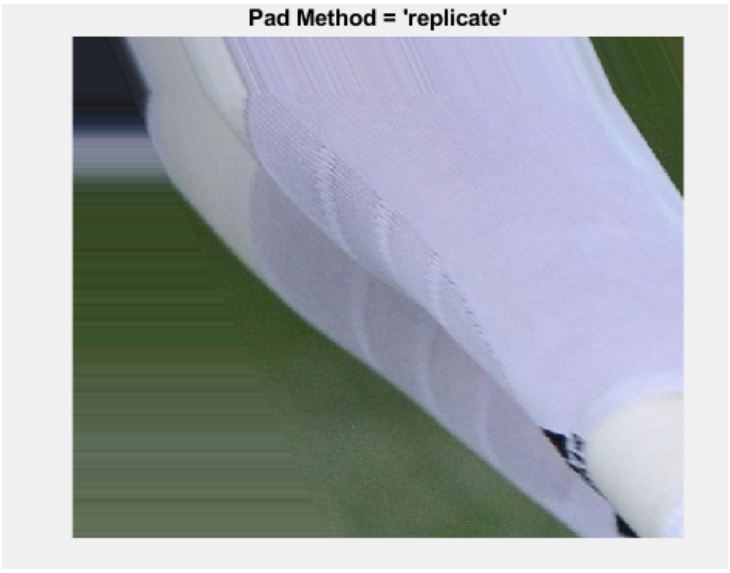
figure, imshow(Bf);
title('Pad Method = ''fill''');

R = makeresampler({'cubic','nearest'},'replicate');
Br = imtransform(A,T,R,'XData',[-49 500],'YData', [-49 400]);

figure, imshow(Br);
title('Pad Method = ''replicate''');

R = makeresampler({'cubic','nearest'}, 'bound');
Bb = imtransform(A,T,R,'XData',[-49 500],'YData',[-49 400],...
                 'FillValues',orange);
figure, imshow(Bb);
title('Pad Method = ''bound''');
```





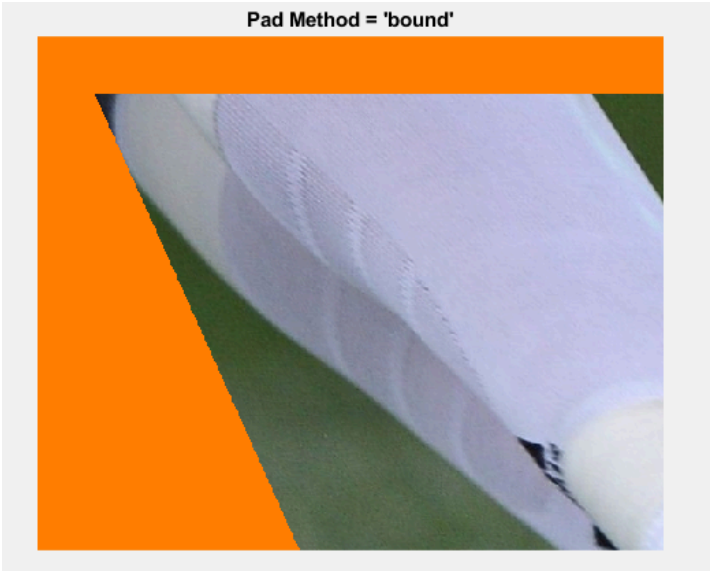


Figure 7: Source Code and Fill, Replicate, and Bound methods for Part 2

The final step was to try out two more padding methods, circular and symmetric methods. The circular method is a circular repetition, where its elements are repeated like a loop. The symmetric padding method uses reflection to fill the padding, rather than repeating the original image, as the circular method does. The figure below shows this clearly.

```
%Step 4
Thalf = maketform('affine',[1 0; a 1; 0 0]/2);

R = makeresampler({'cubic','nearest'},'circular');
Bc = imtransform(A,Thalf,R,'XData',[-49 500],'YData',[-49 400],...
                 'FillValues',orange);
figure, imshow(Bc);
title('Pad Method = ''circular''');

R = makeresampler({'cubic','nearest'},'symmetric');
Bs = imtransform(A,Thalf,R,'XData',[-49 500],'YData',[-49 400],...
                 'FillValues',orange);
figure, imshow(Bs);
title('Pad Method = ''symmetric''');
```





Figure 8: Source Code and Circular and Symmetric methods for Part 2